



Memoria Practica 2

Bases de Datos 2

Álvaro Pérez Gregorio 870097
Mario Recio Lueza 838583



Escuela Universitaria
Politécnica - Teruel

Universidad Zaragoza

Índice

Introducción:	2
Objetivo:	2
Motivación:	2
1. Realiza el diseño de la base de datos que se implementó en el módulo de SQL para MongoDB teniendo en cuenta las distintas formas que existen en Mongo de expresar las relaciones entre colecciones y sus respectivas ventajas/desventajas.	3
2. Realiza un script de automatización de inserción de distintos volúmenes de datos (small, medium, large) y compáralo frente a la base de datos MySQL original.	5
3. Realiza las mismas consultas que se realizaron en la práctica de SQL (si no tienes ninguna consulta que involucre varias tablas, al menos 2-4 de ellas, añádela) y haz una comparativa del rendimiento.....	6
4. Investiga si existe algún tipo de optimización que se pueda realizar para mejorar tanto las inserciones como las consultas realizadas previamente. ¿Podrías aprovecharte del sharing que ofrece MongoDB para particionar la información en distintos shards? ¿Podrías ser menos restrictivo en los ACKs de confirmación de escritura de los replica set que componen tu cluster?	7
5. En los RDMS se respetan las propiedades ACID en las transacciones que se realizan, lo que garantiza la consistencia de la información en todo momento. ¿Existe algo similar en MongoDB?. A d e m á s , ¿ Observas que hay duplicidad de información? ¿De que manera podrías evitarlo tanto a nivel de base de datos cómo a nivel de aplicación?.....	8
6. Conclusiones obtenidas. ¿Crees que tu modelo de datos tiene más sentido en SQL o ves que encaja más MongoDB para él?	9
ANEXO:	10

Introducción:

En esta práctica se nos pide diseñar e implementar una base de datos utilizando un enfoque NoSQL basado en MongoDB. Partiendo de la temática y el modelo de datos desarrollado en la práctica anterior con SQL. Para ello, investigaremos las diferentes formas de representar relaciones entre colecciones en MongoDB, analizando sus ventajas y desventajas frente a las relaciones tradicionales en bases de datos relacionales como MariaDB que es la que utilizamos para la práctica 1.

Además, realizaremos un análisis de rendimiento mediante la inserción de diferentes volúmenes de datos (pequeños, medianos, grandes) a través de un script de automatización y compararemos los tiempos de ejecución con los de la anterior práctica en MariaDB. También ejecutaremos las mismas consultas realizadas en la práctica de SQL, para evaluar también el rendimiento de MongoDB en comparación con un SGBD relacional.

En nuestro caso, continuaremos con la temática de las competiciones de fútbol universitario que explicaremos más adelante.

Objetivo:

El principal objetivo de esta práctica es afianzar los conocimientos adquiridos en la asignatura sobre bases de datos, pero desde una perspectiva NoSQL. Nos enfocaremos en comprender las diferencias fundamentales entre un SGBD relacional y uno NoSQL, explorando cómo se modelan los datos, cómo se representan las relaciones entre tablas y cómo afecta esta estructura al rendimiento del sistema.

También, analizamos las ventajas y desventajas de MongoDB respecto a MariaDB, centrándonos en aspectos como la escalabilidad y flexibilidad del esquema, la velocidad de inserción de datos y la eficiencia en la ejecución de consultas y su optimización

Motivación:

En esta práctica se nos plantea el reto de trasladar un modelo de base de datos relacional a un enfoque NoSQL usando MongoDB. Esto nos permitirá comprender mejor las diferencias de ambos modelos.

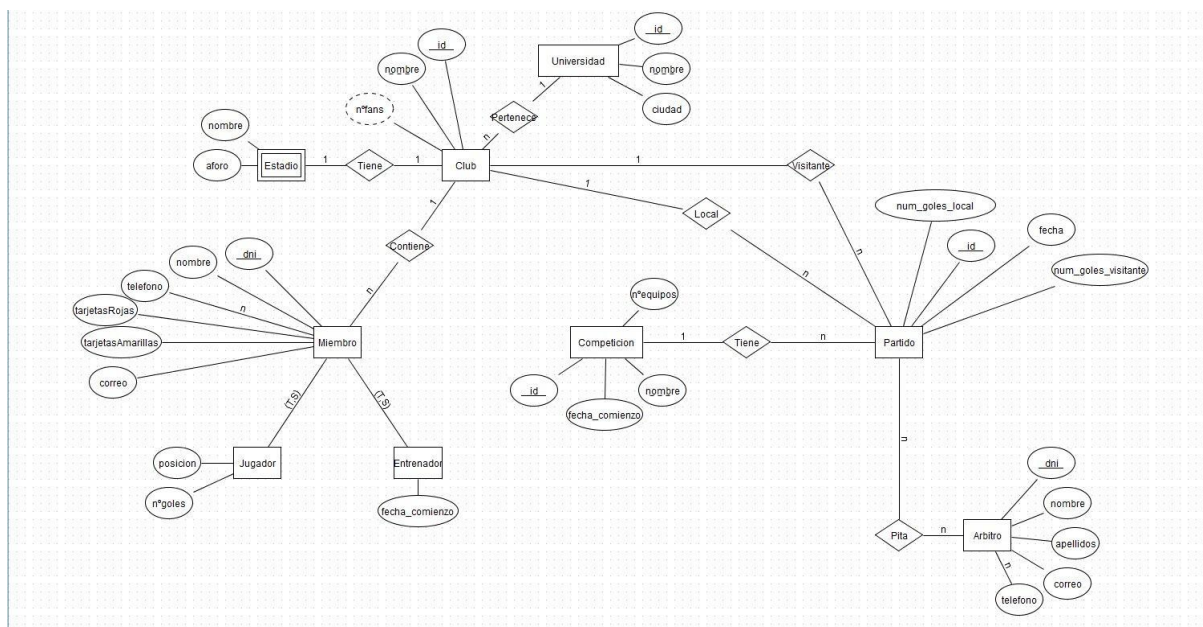
Este ejercicio además nos ayudará a desarrollar una visión más amplia sobre cuándo puede ser conveniente utilizar un modelo relacional o un modelo documental en función de las necesidades del sistema.

1. Realiza el diseño de la base de datos que se implementó en el módulo de SQL para MongoDB teniendo en cuenta las distintas formas que existen en Mongo de expresar las relaciones entre colecciones y sus respectivas ventajas/desventajas.

Para recordar la temática del diseño de la base de datos explicamos a continuación como hicimos en la primera práctica y mostramos también el diseño conceptual usado de plantilla para realizarla:

La base de datos consistirá en tres entidades principales, Club, Miembro y Partido, donde los Clubes pertenecerán a una Universidad y tendrán varios Miembros los cuales cuentan con una generalización especificación total solapada así que podrán ser Jugadores o Entrenadores. El Club también dispondrá de un Estadio pero como se puede ver en el diseño tendrá la clave primaria de Club, es decir, será una entidad débil.

Por otro lado, Partido, competirán dos entidades de Club, pertenecerá a una Competición y lo "pitarán" uno o varios Árbitros. Por la relación Partido-Árbitro debemos crear otra tabla llamada Pita en consecuencia a ser una relación N-N.



Entrando en materia, para transformar dicho modelo en NoSQL, sobre todo enfocándonos en las relaciones entre entidades, hemos tenido que utilizar una serie de aspectos embebidos y de referencias ya que en MongoDB no existen las claves ajenas, es decir que no se puede relacionar las clases tan sencillamente como en MariaDB.

En MongoDB, el esquema de base de datos es flexible, por lo que el desarrollador debe decidir cómo implementar las relaciones según los requisitos que se tengan o se pidan, en concreto existen 3 tipos de relaciones que son: One-to-One, donde un documento suele embeberse dentro de otro, One-to-Many embebido, donde los documentos de la relación se embeben dentro de otro en una estructura de tipo array, y One-to-Many con referencias, donde se utilizan referencias al `_id` de los documentos relacionados, en vez de embeberlos por completo

Ahora explicaremos qué relaciones de cada tipo hemos utilizado para cada una de las de nuestro diseño.

La relación entre una universidad y sus clubes es de tipo **One-to-Many embebido**, ya que los documentos de los clubes están embebidos dentro de cada documento de universidad en un array.

La relación entre un club y su estadio es de tipo **One-to-One embebido**, ya que el documento del estadio está embebido dentro del documento del club.

La relación entre un club y sus miembros es de tipo **One-to-Many embebido**, ya que los documentos de los miembros están embebidos dentro del documento del club en un array.

Dentro de los miembros, la relación entre un miembro y su rol específico (jugador o entrenador) es de tipo **One-to-One embebido**, ya que esta información está embebida directamente dentro del documento del miembro.

La relación entre un partido y la competición (``id_competicion``) es de tipo **One-to-Many con referencias**, ya que se utiliza una referencia al ``id_competicion`` en lugar de embeber el documento completo de la competición.

La relación entre un partido y los equipos locales y visitantes (``id_local`` y ``id_visitante``) es de tipo **One-to-Many con referencias**, ya que se utilizan referencias a los ``id_club`` en lugar de embeber los documentos completos de los equipos.

La relación entre un partido y los árbitros que pitaron (``pitas``) es de tipo **One-to-Many con referencias**, ya que se utilizan referencias a los ``dni_arbi`` en lugar de embeber los documentos completos de los árbitros.

2. Realiza un script de automatización de inserción de distintos volúmenes de datos (small, medium, large) y compáralo frente a la base de datos MySQL original.

Para insertar datos utilizamos un script con el siguiente comando en la terminal **node data_inserter.js --x** ; donde x será small, medium o large dependiendo del tamaño de datos que queramos insertar a la base de datos (también se puede poner como opción clean para limpiar la base de datos antes de realizar una inserción, y de esta manera evitar errores)

Para generar los datos nos hemos empleado **const faker = require('faker')** ; que nos genera unos datos imaginarios, de esta forma es mucho más fácil insertar todo y evitar la duplicidad de datos.

Al igual que en la práctica anterior daremos los tiempos de ejecución de la inserción de datos, pero esta vez comparando los tiempos en MongoDB y MariaDB.

	Small dataset	Medium dataset	Large dataset
MongoDB	0.971 seg	8.323 seg	65.825 seg
≈data insertada	147.086	922.124	4.178.143
MariaDB	1.342 seg	13.366 seg	304.268 seg
≈data insertada	160.850	943.000	3.816.000

Como se puede observar los tiempos de ejecución de MongoDB son, en todos los casos, menores a los de MariaDB, esto se debe a muchos aspectos que comentamos a continuación.

En MariaDB hay índices únicos, claves ajenas y restricciones así que cada inserción debe verificarse antes de efectuarse, en MongoDB al ser NoSQL, la validación de los índices es mucho más flexible y no existen las claves ajenas por lo que se reduce bastante el tiempo de procesamiento, además en Mongo se almacenan datos en un formato similar a JSON llamado BSON lo cual permite insertar documentos sin necesidad de definir una estructura rígida mientras que en MariaDB para cada inserción se debe ajustar al esquema predefinido lo que ralentiza la escritura si hay muchas restricciones o relaciones.

También como curioso MongoDB usa el motor de almacenamiento **WiredTiger** que usa algoritmos para reducir el tamaño de almacenamiento, lo que ahorra espacio y mejora la eficiencia además de que maneja escrituras en caché antes de persistir en disco para reducir cargas y tiempo. En el otro lado de la moneda, MariaDB usa **InnoDB** como motor y este necesita garantizar consistencia y durabilidad en cada escritura por lo que le conllevará más tiempo realizar las operaciones.

3. Realiza las mismas consultas que se realizaron en la práctica de SQL (si no tienes ninguna consulta que involucre varias tablas, al menos 2-4 de ellas, añádela) y haz una comparativa del rendimiento.

1.- La primera consulta mostrará los 10 clubes que han anotado más goles en sus partidos ganados durante el último año.

Los tiempos de ejecución serán 8,323 segundos para **MongoDB** y 13,8759 segundos para **MariaDB** (InnoDB).

2.-Esta consulta muestra todos los partidos en los que un club en particular ha jugado, indicando el rival, la fecha y el resultado, ordenados del más reciente al más antiguo.

Los tiempos de ejecución serán 1,150 segundos para **MongoDB** y 0,205 segundos para **MariaDB** (InnoDB).

3.-Esta consulta cuenta los partidos ganados por cada club (ya sea jugando como local o visitante) y los muestra en orden descendente, con el club más exitoso en la parte superior.

Los tiempos de ejecución serán 9,345 segundos para **MongoDB** y 13,7071 segundos para **MariaDB** (InnoDB).

4.- Esta consulta obtiene un listado de todos los partidos con detalles como la competición, los equipos, el resultado y los árbitros. Si un partido no tiene árbitros, se muestra "Sin árbitros". Los resultados se muestran ordenados del más reciente al más antiguo

Los tiempos de ejecución serán 29,450 segundos para **MongoDB** y 17,1933 segundos para **MariaDB** (InnoDB).

Como se puede observar en los resultados el modelo de MongoDB suele ser más rápido que MariaDB debido a algunos factores clave como índices más eficientes donde Mongo permite índices compuestos, parciales y multidimensionales en estructuras embebidas, pero en Maria los índices funcionan a nivel de tabla y pueden no ser tan eficientes en consultas con muchos filtros. Mongo utiliza un sistema de procesamiento de datos especial llamado *Aggregation Framework*, el cual está diseñado para manejar grandes cantidades de datos con filtros, agrupaciones y ordenaciones mientras que Maria usa "GROUP BY y JOIN" lo que puede ser más costoso en términos de rendimiento.

4. Investiga si existe algún tipo de optimización que se pueda realizar para mejorar tanto las inserciones como las consultas realizadas previamente. ¿Podrías aprovecharte del sharing que ofrece MongoDB para particionar la información en distintos shards? ¿Podrías ser menos restrictivo en los ACKs de confirmación de escritura de los replica set que componen tu cluster?

Existen varias optimizaciones que se pueden realizar para mejorar tanto las inserciones como las consultas en MongoDB. Estas incluyen la creación de índices adecuados, el uso de operaciones en lote, la implementación de proyecciones selectivas, la desnormalización de datos cuando sea necesario, y la colocación de etapas \$match tempranas en los pipelines de agregación.

El sharding en MongoDB puede ser aprovechado para particionar la información en distintos shards. Esto ayuda a distribuir la carga de trabajo y mejorar la escalabilidad y el rendimiento general de la base de datos. La selección correcta de la clave de shard es crucial para asegurar una distribución uniforme de los datos.

Es posible ser menos restrictivo en los ACKs de confirmación de escritura de los replica sets. Al ajustar el nivel de write concern, se puede mejorar el rendimiento de las escrituras, aunque esto puede introducir un mayor riesgo de pérdida de datos. Niveles menos restrictivos incluyen write concern configurado a w: 1 o incluso w: 0, dependiendo del balance deseado entre rendimiento y durabilidad.

5. En los RDMS se respetan las propiedades ACID en las transacciones que se realizan, lo que garantiza la consistencia de la información en todo momento. ¿Existe algo similar en MongoDB?. Además, ¿Observas que hay duplicidad de información? ¿De que manera podrías evitarlo tanto a nivel de base de datos cómo a nivel de aplicación?

MongoDB soporta propiedades ACID en las transacciones. Desde la versión 4.0, MongoDB proporciona soporte completo para transacciones ACID a nivel de documento y colecciones únicas, y desde la versión 4.2, soporta transacciones distribuidas que abarcan múltiples shards.

En MongoDB puede haber duplicidad de información debido a la desnormalización de datos, que se emplea para mejorar el rendimiento de las consultas.

Maneras de evitar la duplicidad:

A nivel de base de datos:

- Utilizar índices únicos para evitar la inserción de duplicados.
- Implementar validaciones de esquema para asegurar que los documentos cumplan con ciertos criterios antes de ser insertados o actualizados.

A nivel de aplicación:

- Implementar mecanismos de control de concurrencia para evitar la inserción de duplicados.
- Utilizar operaciones atómicas para asegurarse de que las actualizaciones se realicen de manera segura sin duplicar datos.

6. Conclusiones obtenidas. ¿Crees que tu modelo de datos tiene más sentido en SQL o ves que encaja más MongoDB para él?

Como conclusiones podemos observar a lo largo de toda la práctica que MongoDB es más rápido en la inserción de datos para la base de datos ya que usa un modelo sin esquema, con escrituras en memoria antes de volcar a disco.

MongoDB también supera a MariaDB en el tiempo de ejecución de las consultas, principalmente las que requieren agregaciones, búsquedas en grandes volúmenes de datos y estructuras flexibles.

MongoDB será más rápido en general, pero MariaDB tiene ventajas en transacciones y consistencia, es decir los datos son altamente relacionales con integridad referencial clara.

Finalmente nos queda la pregunta de cuál elegir ya que si trabajamos grandes volúmenes de datos que requieren inserciones flexibles y rápidas, MongoDB nos daría la mejor solución, pero por otro lado si lo que nos interesa es un mejor manejo de las relaciones estructuradas y una mejor organización y consistencia, MariaDB sería la indicada.

Para decantarnos por un modelo de los dos, elegiremos para nuestro diseño por el momento SQL ya que le damos más importancia a la consistencia de datos que a la cantidad de volumen de datos, principalmente por gusto personal pero también porque consideramos que es mejor que las relaciones sean con una firmeza adecuada por la facilidad que te sugiere. Por otro lado, si la idea de nuestro ejercicio es la de “fútbol universitario” no necesitaríamos volúmenes de datos inmensos.

Como toque final esta elección nos deja con la idea de elegir “la calidad(facilidad y consistencia) antes que la cantidad(rapidez y grandes volúmenes de datos)”.

ANEXO:

El script de inserción de datos en la base de datos es el mismo tanto para la dataset small, medium o large, simplemente se añadirá un parametro de opcion detrás del comando de ejecución como se ha explicado en el punto correspondiente anterior. Tanto el script como todas las consultas están adjuntadas junto a esta memoria en el trabajo.